



Что мы будем делать

Сейчас я построю небольшой, но полностью настоящий GitHub-процесс.

У нас уже существует локальный проект:

```
<каталог проекта>
```

Сначала я проверю GitHub CLI и авторизацию. Затем создам или подключу удалённый GitHub-репозиторий. После этого подготовлю GitHub Actions и проведу изменение кода через рабочую ветку и Pull Request.

Только после этого я передам работу Hermes. Агент должен будет создать отдельную ветку, выполнить небольшое изменение, проверить его локально, создать commit, отправить ветку на GitHub и открыть Pull Request.

После создания PR работа агента ещё не закончится. Он должен получить состояние CI непосредственно с GitHub и сообщить, выполняется проверка, завершилась успешно или упала.

Проверяем GitHub CLI

Открываю Hermes Desktop и нужный Project:

```
<имя Project>
```

Справа открываю Terminal сочетанием:

```
Ctrl + `
```

Проверяю текущую директорию:

```
Get-Location
```

Сохраняю корни проекта и Git-репозитория в переменные:

```
$projectRoot = (Get-Location).Path  
$repoRoot = (git rev-parse --show-toplevel).Trim()
```

Теперь проверяю GitHub CLI:

```
gh --version
```

Если отображается версия, продолжаю.

Если Windows сообщает, что команда `gh` не найдена, открываю обычный PowerShell и выполняю:

```
winget install --id GitHub.cli --source winget
```

После установки полностью закрываю окно терминала и открываю новое. Официальная инструкция GitHub отдельно предупреждает, что установщик изменяет `PATH`, поэтому простого открытия новой вкладки Windows Terminal может быть недостаточно.

После этого снова проверяю:

```
gh --version
```

Авторизуем GitHub CLI

Теперь выполняю:

```
gh auth status
```

Если авторизация уже есть, GitHub CLI покажет активный аккаунт.

Если авторизации нет, выполняю:

```
gh auth login
```

Дальше появляется интерактивный выбор.

Я выбираю:

```
GitHub.com
```

Затем выбираю:

```
HTTPS
```

На вопрос об аутентификации Git выбираю вариант с использованием учётных данных GitHub.

Для входа выбираю:

```
Login with a web browser
```

GitHub CLI покажет одноразовый код и предложит открыть браузер.

Нажимаю Enter.

В браузере открывается GitHub. Если аккаунт ещё не авторизован, выполняю вход.

В поле устройства ввожу код, который показал `gh`.

Нажимаю кнопку подтверждения авторизации.

Возвращаюсь в PowerShell и снова выполняю:

```
gh auth status
```

GitHub CLI официально рекомендует именно эту проверку перед работой с PR, и Hermes GitHub Skill также выполняет `gh auth status` как предварительный этап.

Проверяем локальный Git

Работаю из уже открытого корня проекта. Git-команды автоматически находят родительский репозиторий; при необходимости его путь доступен в `$repoRoot`.

Проверяю текущую ветку:

```
git branch --show-current
```

Имя основной ветки определяю из настроек удалённого репозитория. В примерах это может быть:

```
main
```

Если текущая ветка называется `master`, можно переименовать:

```
git branch -M main
```

Теперь:

```
git status
```

Перед настройкой удалённого репозитория проверяю состав рабочего дерева и осознанно выбираю изменения, которые относятся к текущей задаче.

Проверяю:

```
git status
```

Состав будущего изменения проверяю по выводу перед фиксацией истории.

Проверяем наличие GitHub remote

Выполняю:

```
git remote -v
```

Если уже есть:

```
origin
```

с адресом GitHub, новый репозиторий создавать не нужно.

Дополнительно можно проверить:

```
gh repo view
```

Если GitHub CLI правильно определяет репозиторий, переходим дальше.

Если `origin` отсутствует, создадим учебный репозиторий прямо через CLI.

Создаём GitHub-репозиторий, если его ещё нет

Выполняю из корня Git-репозитория:

```
Set-Location $repoRoot  
gh repo create hermes-multiagent-lab --private --source . --remote origin --push  
Set-Location $projectRoot
```

GitHub CLI создаст приватный репозиторий в текущем аккаунте, добавит его как `origin` и отправит текущую ветку.

Проверяю:

```
git remote -v
```

Затем:

```
gh repo view
```

Если хочу открыть репозиторий в браузере:

```
gh repo view --web
```

Откроется страница GitHub.

Теперь наш локальный Git связан с реальным Git-хостингом.

Создаём отдельный материал для CI

Чтобы результат занятия не зависел от состояния всех предыдущих файлов проекта, я сделаю маленький изолированный пример.

В Hermes Desktop возвращаюсь в центральный чат и отправляю:

В текущем проекте создай каталог ci_lab.

В нём создай файл text_utils.py со следующим содержимым:

```
def normalize_username(name: str) -> str:
    return name.strip().lower()
```

Также создай каталог ci_lab/tests и файл ci_lab/tests/test_text_utils.py:

```
import unittest

from ci_lab.text_utils import normalize_username

class TextUtilsTests(unittest.TestCase):

    def test_trim_and_lowercase(self):
        self.assertEqual(
            normalize_username(" Alice "),
            "alice",
        )

if __name__ == "__main__":
    unittest.main()
```

Никаких других файлов не изменяй.

Теперь создаю пустой:

```
ci_lab/__init__.py
```

и:

```
ci_lab/tests/__init__.py
```

Проверяю локально:

```
.\.venv\Scripts\python.exe -m unittest discover -s ci_lab\tests -v
```

Тест должен пройти.

Создаём GitHub Actions workflow

Workflow GitHub Actions всегда находится в `.github/workflows` относительно корня Git-репозитория. В нашей структуре учебный Python-проект вложен в общий репозиторий, поэтому сам workflow находится рядом с каталогом `hermes-multiagent`, а команда тестирования получает явную рабочую директорию.

В центральном чате Hermes отправляю:

Создай файл:

```
.github/workflows/ci.yml
```

со следующим содержимым.

Другие файлы не изменяй.

Содержимое:

```
name: CI

on:
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  tests:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: hermes-multiagent

    steps:
      - name: Получить исходный код
        uses: actions/checkout@v7
```

```
- name: Настроить Python
  uses: actions/setup-python@v7
  with:
    python-version: "3.13"

- name: Запустить тесты CI-практики
  run: python -m unittest discover -s ci_lab/tests -v
```

Здесь ключевая часть:

```
on:
  pull_request:
```

означает, что workflow будет реагировать на события Pull Request.

GitHub рекомендует для Python workflows использовать `actions/checkout`, `actions/setup-python` и затем запускать обычную команду тестирования проекта.

Проверяем подготовленные изменения

CI и учебный пакет подготовлены для включения в рабочую ветку вместе с изменением `normalize_username`. Форма истории выбирается по правилам проекта.

В Terminal выполняю:

```
git status --short
```

Проверяю:

```
git status
```

Проверяю, что состав подготовленных изменений соответствует задаче. Дальнейшие операции Git выполняются в рамках выбранного PR-процесса проекта.

Проверяем Actions через интерфейс GitHub

Открываю репозиторий:

```
gh repo view --web
```

В браузере в верхнем меню репозитория нахожу вкладку:

Actions

Нажимаю её.

До публикации feature-ветки новый workflow может ещё не отображаться. После создания Pull Request в списке появится:

```
CI
```

Сейчас отсутствие запуска может быть нормальным, потому что наш workflow настроен только на:

```
pull_request
```

Он запустится после создания PR.

Проверяем встроенный GitHub Skill Hermes

Возвращаюсь в Hermes Desktop.

Создаю новый чат, чтобы старые экспериментальные инструкции не влияли на работу.

В поле сообщения можно проверить доступность GitHub Skill через slash-команду:

```
/github-pr-workflow
```

Hermes регистрирует установленные Skills как slash-команды; встроенный `github-pr-workflow` предназначен для полного жизненного цикла PR: branch, commit, open PR, CI и merge.

Если команда загрузилась, Hermes уже имеет специализированные инструкции по GitHub workflow.

Но конкретную задачу я пока не даю.

Создаём задачу для Pull Request

Наша задача будет небольшой.

Исходная функция:

```
def normalize_username(name: str) -> str:
    return name.strip().lower()
```

Мы хотим, чтобы пробелы внутри имени пользователя превращались в символ .

Например:

```
" Alice Smith "
```

должно превращаться в:

```
alice_smith
```

Агент также должен добавить тест этого поведения.

Это достаточно маленькое изменение, чтобы основное внимание занятия осталось на PR и CI.

Передаём задачу Hermes

В новом чате Hermes отправляю:

Работаем с текущим GitHub-репозиторию.

Выполни задачу полностью через GitHub PR workflow.

Требование к коду:

Функция `ci_lab/text_utils.py`:

```
normalize_username(name)
```

должна после удаления внешних пробелов и преобразования строки в нижний регистр также заменять внутренние пробелы символом подчёркивания.

Пример:

```
" Alice Smith "
```

должно стать:

```
"alice_smith"
```

Добавь тест этого поведения.

Ожидаемый результат GitHub-процесса:

сначала проверь `gh auth status` и состояние репозитория;

определи основную ветку репозитория и подготовь рабочую ветку для изменения;

выполни только необходимые изменения;

запусти локальные тесты;

проверь `git diff`;

создай `commit` с понятным сообщением;

```
отправь ветку в origin;  
  
создай Pull Request в основную ветку через GitHub CLI;  
  
после создания PR получи его номер и URL;  
  
после открытия PR отдельно проверь состояние CI через gh pr checks;  
  
в конце сообщи:  
  
имя ветки;  
commit;  
номер PR;  
URL PR;  
текущий статус CI.  
  
Не утверждай, что CI прошёл успешно, пока это не подтверждено GitHub.
```

Теперь Hermes должен выполнить последовательность команд самостоятельно.

Я не ввожу эти Git-команды вручную. В этом и заключается практическая часть: агент работает как исполнитель PR-процесса.

Наблюдаем первые действия агента

Я слежу за карточками Terminal Tool в Hermes.

Сначала ожидаю проверку:

```
gh auth status
```

Затем:

```
git status
```

и определение текущей ветки.

Hermes GitHub Skill прямо требует предварительно проверять `gh auth status`, а удалённое состояние получать свежими вызовами `gh`, а не предполагать его.

После этого агент должен определить основную ветку, при необходимости получить её актуальную версию и подготовить рабочую ветку.

Я ожидаю что-то эквивалентное:

```
$baseBranch = gh repo view --json defaultBranchRef --jq '.defaultBranchRef.name'
```

```
git switch $baseBranch
```

```
git pull --ff-only
```

```
$workBranch = "feature/normalize-username"  
git switch -c $workBranch
```

Названия фактических команд и веток могут различаться. Нас интересует результат: изменение происходит через рабочую ветку и Pull Request.

Проверяем создание ветки в интерфейсе Hermes

После того как агент сообщил о создании ветки, открываю справа Terminal и самостоятельно выполняю:

```
git branch --show-current
```

Должно отображаться имя выбранной рабочей ветки, например:

```
feature/normalize-username
```

Теперь:

```
git status
```

Во время работы Coder могут появляться изменённые файлы.

Наблюдаем изменение и локальную проверку

Hermes должен изменить:

```
ci_lab/text_utils.py
```

и:

```
ci_lab/tests/test_text_utils.py
```

После изменений ожидаю запуск:

```
python -m unittest discover -s ci_lab/tests -v
```

или эквивалентной команды через `.venv`.

Важно, чтобы локальный тест прошёл до создания PR.

Затем агент должен посмотреть:

```
git diff
```

Только после анализа diff он должен создать commit.

Проверяем commit

После того как Hermes сообщил о commit, в Terminal выполняю:

```
git log -1 --oneline
```

Я должен увидеть новый commit.

Например:

```
abc1234 Add username space normalization
```

Затем:

```
git status
```

Рабочая директория должна быть чистой.

Наблюдаем push

Hermes должен отправить ветку:

```
feature/normalize-username
```

в:

```
origin
```

Обычно для первого push используется:

```
git push -u origin feature/normalize-username
```

После этого локальная ветка получает upstream.

Проверяю:

```
git branch -vv
```

Возле текущей ветки должна отображаться связь с:

```
origin/feature/normalize-username
```

Наблюдаем создание Pull Request

Следующим действием Hermes должен использовать GitHub CLI.

GitHub CLI официально создаёт PR через:

```
gh pr create
```

Для указания целевой и исходной веток используются `--base` и `--head`; заголовок и описание можно передать через `--title` и `--body`.

Например, агент может выполнить команду примерно такого вида:

```
gh pr create --base main --head feature/normalize-username --title "Normalize spaces in usernames" --body "..."
```

После успешного создания `gh` возвращает URL Pull Request.

Проверяем PR вручную через GitHub

После того как Hermes сообщает URL, я нажимаю ссылку в его ответе.

Если ссылка не открывается из интерфейса, выполняю:

```
gh pr view --web
```

GitHub CLI официально поддерживает `gh pr view --web` для открытия PR текущей ветки в браузере.

В браузере я оказываюсь на странице Pull Request.

В верхней части вижу заголовок PR.

Под ним вижу направление:

```
feature/normalize-username
```

В:

```
main
```

Ниже открываю вкладку:

```
Files changed
```

Проверяю, что изменены только нужные файлы.

Возвращаюсь на:

```
Conversation
```

В нижней части страницы должен появиться блок автоматических checks.

Что происходит сразу после открытия PR

GitHub видит событие:

```
pull_request
```

и запускает `.github/workflows/ci.yml`.

Поэтому первые несколько секунд или минут статус может быть не зелёным и не красным, а:

```
queued
```

или:

```
in progress
```

Это нормальное состояние.

Задача агента — не угадать итог, а получить реальный серверный статус.

Hermes проверяет CI

После создания PR в нашем первоначальном промпте уже было требование получить checks.

Hermes должен вызвать:

```
gh pr checks
```

Если номер PR известен, может использовать:

```
gh pr checks <номер>
```

Команда `gh pr checks` специально показывает CI-состояние одного Pull Request. Она различает состояния `pass`, `fail`, `pending`, `skipping` и `cancel`. Если аргумент не передан, CLI использует PR текущей ветки.

Если CI ещё выполняется, правильный ответ агента должен быть примерно:

```
CI: pending
```

а не:

```
CI успешно пройден.
```

Просим Hermes дождаться завершения checks

Если в первом ответе CI ещё находится в состоянии pending, отправляю следующее сообщение:

Продолжи работу с тем же Pull Request.

Не изменяй код.

Отслеживай CI до завершения через GitHub CLI.

После завершения сообщи фактический результат checks.

Hermes может использовать:

```
gh pr checks --watch
```

Эта команда остаётся активной до завершения проверок. По умолчанию состояние обновляется каждые 10 секунд; также поддерживается `--fail-fast`.

После завершения я ожидаю один из двух реальных результатов:

```
pass
```

или:

```
fail
```

Проверяем CI самостоятельно

Теперь я не ограничиваюсь сообщением агента.

В Hermes Terminal выполняю:

```
gh pr checks
```

Если всё прошло правильно, вижу успешный check workflow **CI**.

Можно получить более структурированный вывод:

```
gh pr checks --json name,state,bucket,workflow
```

Поле **bucket** особенно удобно, потому что GitHub CLI нормализует множество внутренних состояний в категории:

```
pass
fail
pending
skipping
cancel
```

Открываем checks в браузере

Для визуальной демонстрации студентам выполняю:

```
gh pr checks --web
```

Браузер открывает страницу проверок PR.

Либо на странице самого Pull Request нахожу блок:

Checks

и нажимаю:

Details

возле workflow.

GitHub переносит меня в GitHub Actions.

Здесь можно раскрыть job:

```
tests
```

Затем увидеть отдельные steps:

```
Checkout repository
Set up Python
Run tests
```

Таким образом, мы видим не просто зелёную галочку, а фактический серверный процесс.

Проверяем workflow через вкладку Actions

Есть ещё один способ.

На странице репозитория в верхнем меню нажимаю:

```
Actions
```

В списке workflows выбираю:

```
CI
```

Нахожу запуск, связанный с нашей веткой или Pull Request.

Открываю его.

Внутри нажимаю job:

```
tests
```

Теперь вижу полный лог CI.

Это особенно полезно, если checks завершились с ошибкой.

Получаем статус PR целиком

Возвращаюсь в Hermes и отправляю:

```
Получи текущее состояние созданного Pull Request через GitHub CLI.

Покажи:

номер;
```

```
state;  
base branch;  
head branch;  
URL;  
состояние CI.
```

На этом шаге только собери и сообщи текущее состояние.

Hermes может использовать:

```
gh pr view
```

и:

```
gh pr checks
```

`gh pr status` также показывает краткую сводку по PR, включая состояние CI и review.

Специально создаём падающий CI

Теперь проверяю не только успешную, но и диагностическую ветку процесса. В рабочей ветке текущего Pull Request прошу Hermes намеренно изменить ожидаемое значение нового теста так, чтобы тест гарантированно падал.

Отправляю:

```
В текущем Pull Request специально измени ожидаемое значение нового теста так,  
чтобы тест гарантированно падал. Например, ожидай alice-smith вместо  
alice_smith.
```

```
Запусти тест локально и зафиксируй фактическое падение. Затем сохрани изменение  
в истории рабочей ветки и отправь её в удалённый репозиторий. После обновления  
Pull Request дождись результата нового CI и сообщи его, пока не исправляя  
причину.
```

После обновления ветки GitHub автоматически запускает workflow повторно.

Исследуем серверную причину ошибки

Проверяю состояние:

```
gh pr checks --watch
```

CI должен завершиться ошибкой. Затем поручаю Hermes получить доказательства с GitHub, а не выводить причину только из локального запуска:

CI текущего Pull Request завершился ошибкой.

Используя GitHub CLI, определи, какая серверная проверка упала и почему. Покажи фактическую причину на основании GitHub Actions и пока не исправляй код.

Агент может использовать `gh pr checks`, `gh run list`, `gh run view --log-failed` и `gh run watch`. В журнале ожидаю `assertion failure` с различием между `alice_smith` и `alice-smith`.

Исправляем PR через Hermes

После подтверждения причины отправляю:

Исправь намеренно сломанный тест текущего Pull Request.

После изменения запусти локальные тесты, проверь diff, сохрани исправление в рабочей ветке и отправь её в удалённый репозиторий. Используй существующий Pull Request. Дождись нового CI и сообщи итоговый статус GitHub checks.

Способ организации истории выбирается по правилам конкретного проекта: это может быть отдельный исправляющий commit, amend, rebase или squash при merge. Существенен наблюдаемый цикл `ошибка → диагностика → исправление → зелёный CI`.

Завершаем Pull Request

После зелёных проверок Pull Request можно объединить принятой в проекте стратегией. Затем обновляю локальную основную ветку и проверяю, что более свежая версия изменения не осталась только в рабочей ветке:

```
$baseBranch = gh repo view --json defaultBranchRef --jq '.defaultBranchRef.name'
git switch $baseBranch
git pull --ff-only
```

Рабочую ветку после успешного объединения можно удалить локально и удалённо. Конкретная стратегия merge и форма истории не фиксируются этой практикой.

Почему мы не заканчиваем после `gh pr create`

Теперь я фиксирую основную идею занятия.

Команда:

```
gh pr create
```

говорит только о том, что объект Pull Request успешно создан на GitHub. Она ничего не доказывает о качестве кода.

Даже успешный локальный тест не подтверждает серверный CI.

И только:

```
gh pr checks
```

даёт агенту текущее состояние проверок GitHub.

Это соответствует дисциплине встроенного GitHub Skill Hermes: удалённое состояние необходимо читать свежим запросом, а не выводить из локального состояния или прошлого сообщения.

Какой результат должен получиться

Hermes проводит изменение через рабочую ветку и Pull Request, выполняет локальную проверку, получает серверное состояние через `gh pr checks`, наблюдает намеренно созданное падение CI, читает журнал GitHub Actions, исправляет причину и подтверждает новый зелёный запуск.

После принятого в проекте способа объединения основная ветка содержит итоговое изменение, а рабочая ветка не остаётся единственным местом с более свежей версией. Локальный Git, удалённый Pull Request и внешний CI образуют единый проверяемый процесс.